

UNIVERSITA' DEGLI STUDI DI BOLOGNA - CORSO DI LAUREA IN INFORMATICA
 PROVA SCRITTA DI SISTEMI OPERATIVI
 ANNO ACCADEMICO 2023/2024
 10 settembre 2024

Esercizio -I: Essere iscritti su AlmaEsami per svolgere questa prova.

Esercizio 0: Scrivere correttamente nome, cognome, matricola e posizione in tutti i fogli prima di svolgere ogni altro esercizio. Scrivere esclusivamente a penna senza abrasioni. E' vietato l'uso delle penne cancellabili, della matita, dei coprenti bianchi per la correzione (bianchetto) e la scrittura in colore rosso (riservato alla correzione).

Il compito e' formato da tre fogli, sei facciate compresa questa. Le soluzioni che si vogliono sottoporre per la correzione devono essere scritte negli spazi bianchi di questi fogli. Non verranno corretti altri supporti.

E' obbligatorio consegnare il compito, e' possibile chiedere che esso non venga valutato scrivendo "NON VALUTARE" in modo ben visibile nella prima facciata.

Per svolgere questo compito occorre solo una penna e un documento di identità valido. La consultazione o anche solo la disponibilità di altro materiale comporterà l'annullamento del compito (verrà automaticamente valutato gravemente insufficiente).

Esercizio c.1: Implementare il monitor `firstlast` che ha 3 procedure entry:

```
void addfirst(int val);
void addlast(int val);
int getboth(void);
```

Quando un processo chiama la procedure entry `getboth` deve attendere che ci sia almeno un processo che ha chiamato la `addfirst` e uno che ha chiamato la `addlast`. La funzione `getboth` deve sbloccare il *primo* processo fra quelli in attesa per aver chiamato `addfirst` e l'*ultimo* fra quelli in attesa per aver chiamato `addlast`. Il valore di ritorno della `getboth` è la somma dei parametri passati alla `addfirst` e `addlast` dai processi sbloccati.

I processi che chiamano la `addfirst` o la `addlast` devono attendere di essere riattivati da una chiamata `getboth`.

Esercizio c.2: Dato un servizio di message passing asincrono scrivere un servizio di message passing asincrono alternato (senza fare uso di processi server).

```
void altsend(msg_t msg, pid_t destination)
msg_t altrecv(pid_t sender)
```

La prima chiamata di `altrecv` deve restituire il primo messaggio fra quelli in attesa (spediti dal mittente `sender` o da qualsiasi processo se `sender==ANY`). La seconda chiamata di `altrecv` deve restituire l'ultimo fra i messaggi in attesa, la terza chiamata deve restituire il primo, la quarta chiamata l'ultimo e così via.

Esercizio g.1: Si chiede di mostrare il funzionamento dell'Algoritmo del Banchiere bivaluta mostrando quali sia lo stato e l'azione intrapresa in seguito ad ogni richiesta o restituzione di risorse.

Il capitale iniziale è (10,10) cioè 10 per la prima valuta e 10 per la seconda.

Esistono 3 processi P1, P2 e P3 che hanno rispettivamente credito massimo (8,5), (5,8) e (5,5). A partire dalla situazione iniziale nella quale nessun processo ha risorse assegnate mostrare come si comporta l'algoritmo a seguito di ogni operazione della sequenza qui indicata:

- P1 richiede (4,5)
- P3 richiede (3,4)
- P3 termina
- P2 richiede (3,4)
- p1 restituisce (0,3)

Esercizio g.2: rispondere alle seguenti domande (*motivando opportunamente le risposte!*):

- a) In RAID61 (RAID1 di RAID6) qual è il livello di fault tolerance (quanti dischi possono guastarsi? E in RAID16 (RAID 6 di RAID 1) ?
- b) Nella configurazione di un sistema l'amministratore decide di creare una partizione di un disco rotazionale con file system FAT da montare come /tmp. E' una buona o cattiva idea? Perché?
- c) Quali sono i benefici della gestione degli interrupt nidificati e quali sono le difficoltà da affrontare per sviluppare un sistema operativo che supporti gli interrupt nidificati?
- d) perché aumentando la dimensione dell'area di memoria secondaria usata dalla memoria virtuale si corre rischio di trashing?